

Chapter 4

The Processor (Part 1)

Outline

- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme

Introduction

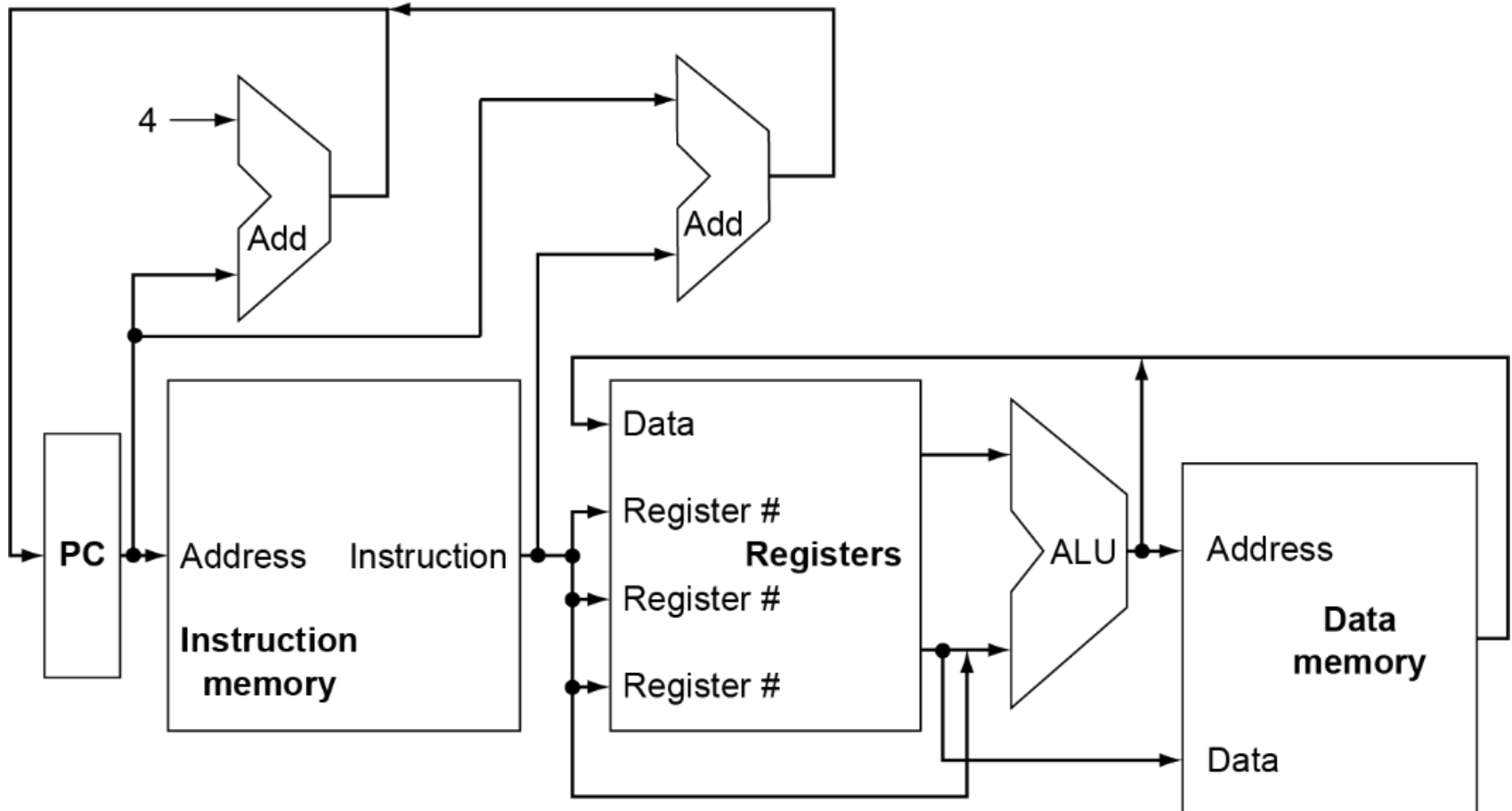
- CPU performance factors
 - Instruction count
 - Determined by ISA and compiler
 - CPI (clock cycles per instruction) and clock cycle time
 - Determined by processor hardware
- Will examine two RISC-V implementations
 - A simplified version (single cycle)
 - A more realistic pipelined version
- A simple RISC-V instruction subset, showing most aspects
 - Memory reference: **ld**, **sd**
 - Arithmetic/logical: **add**, **sub**, **and**, **or**
 - Control transfer: **beq**

Instruction Execution

- PC → instruction memory, fetch instruction
- Register numbers → register file, read registers
- Depending on instruction class
 - Use ALU to calculate
 - Arithmetic/logical result
 - Memory address for load/store
 - Branch comparison
 - Access data memory for load/store
 - Write arithmetic/logical result to a register
 - $PC \leftarrow \text{target address or } PC + 4$

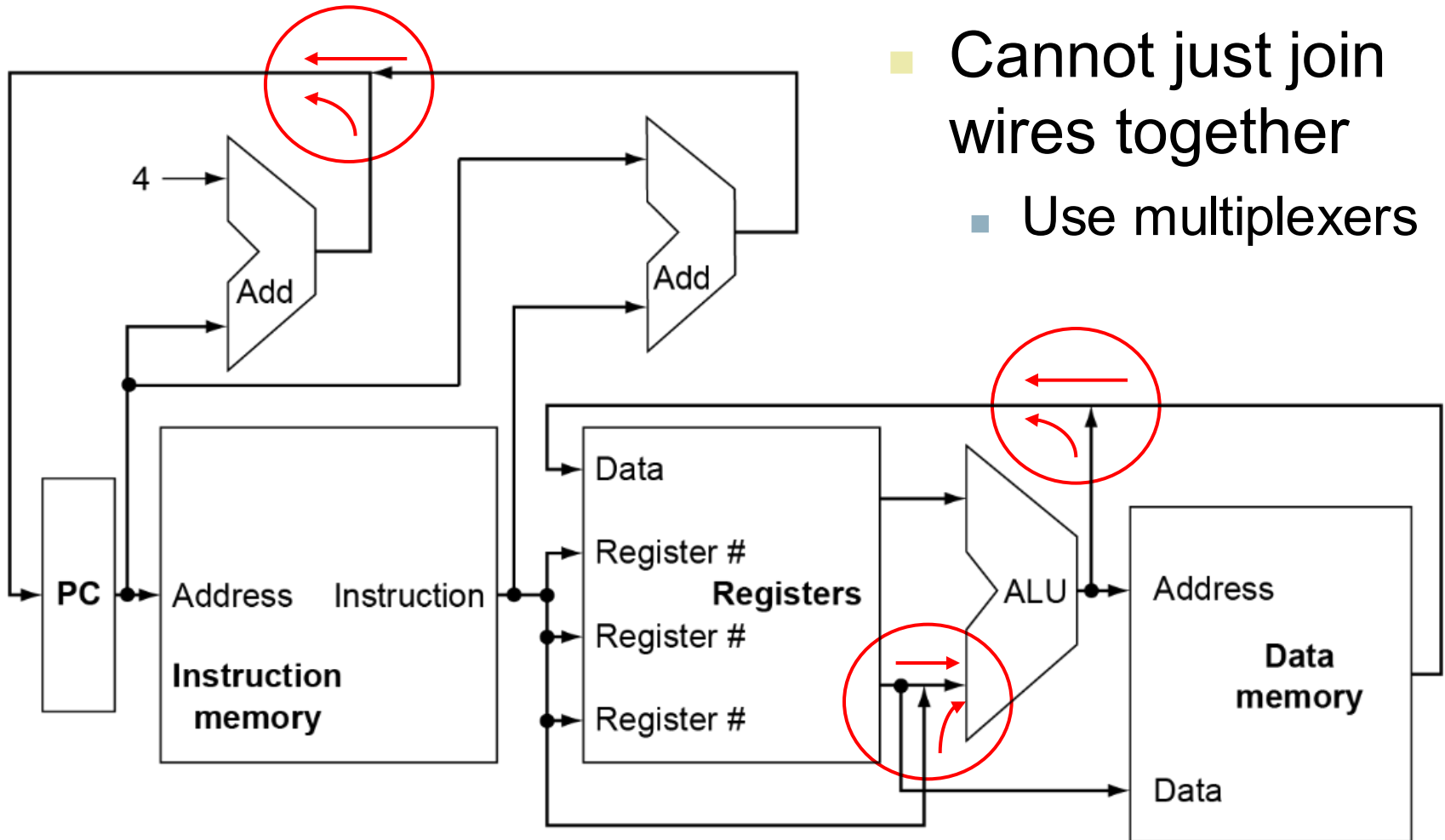
High-level View of a RISC-V Implementation

- Focus: functional units + interconnections

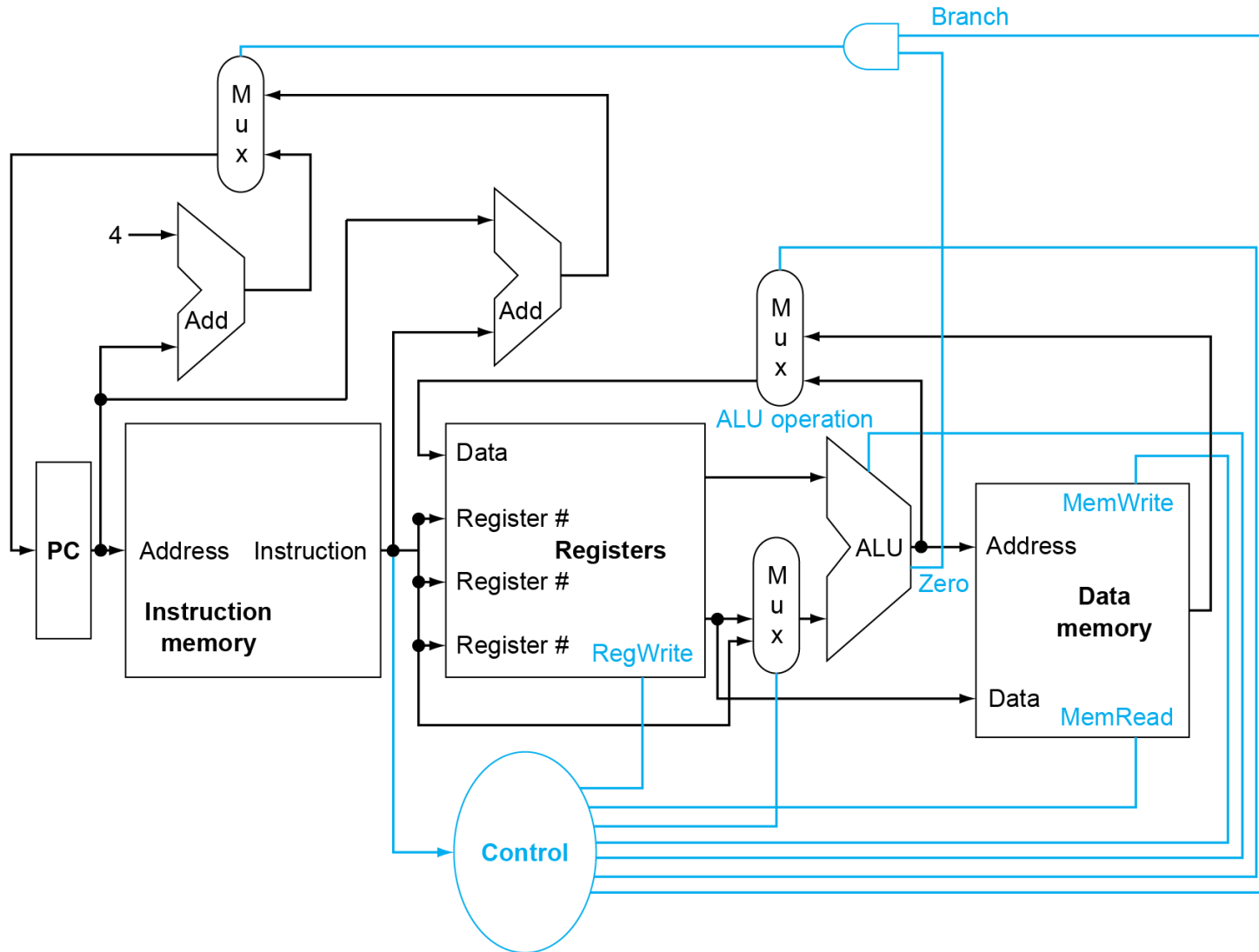


What Is Missing: Multiplexers

- Cannot just join wires together
 - Use multiplexers



What Is Missing: Control



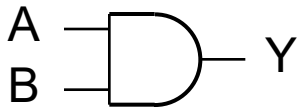
Logic Design Basics

- Information encoded in binary
 - Positive logic: low voltage = 0
high voltage = 1
 - One wire per bit
 - Multi-bit data encoded on multi-wire buses
- Combinational element
 - Operate on data
 - Output is a function of input
- State (sequential) elements
 - Store information

Combinational Elements

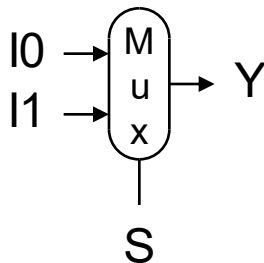
- AND-gate

- $Y = A \& B$



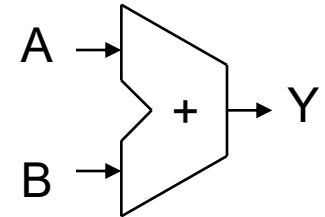
- Multiplexer

- $Y = S ? I1 : I0$



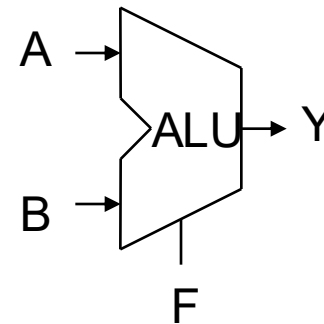
- Adder

- $Y = A + B$



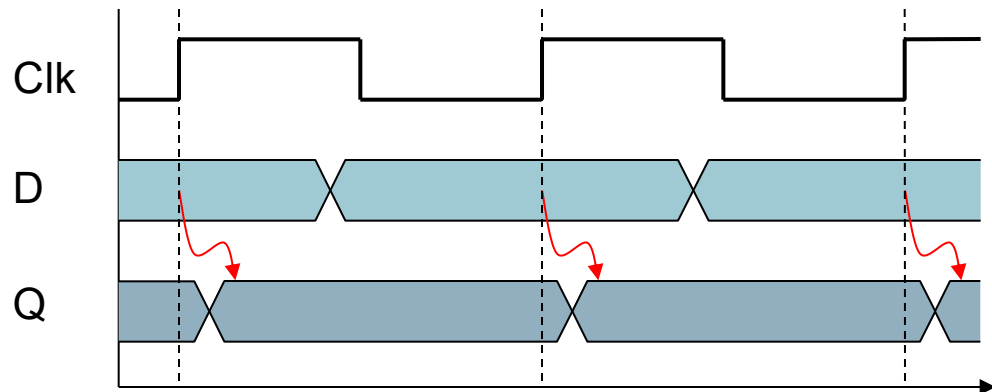
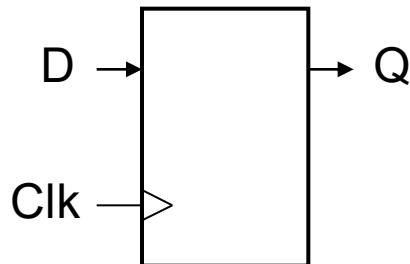
- Arithmetic/Logic Unit

- $Y = F(A, B)$



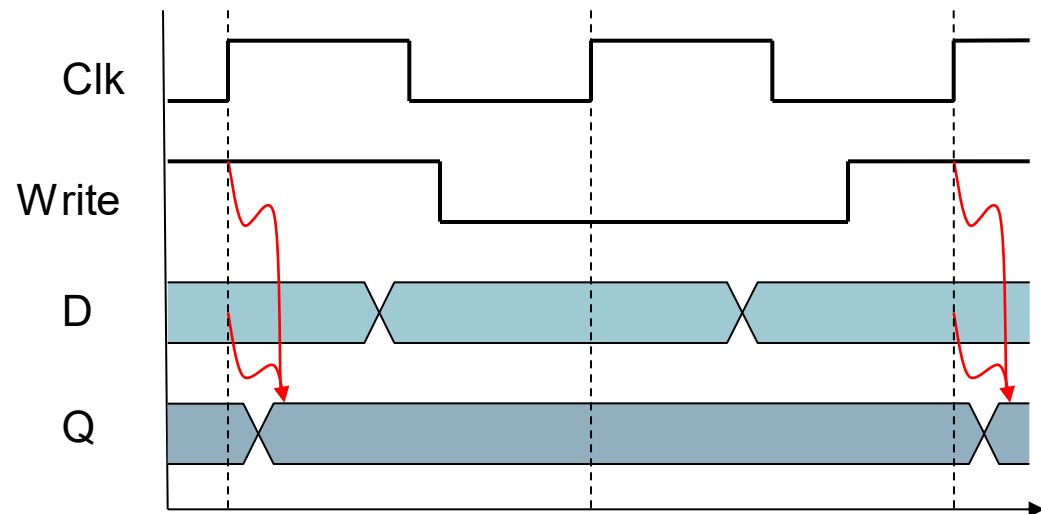
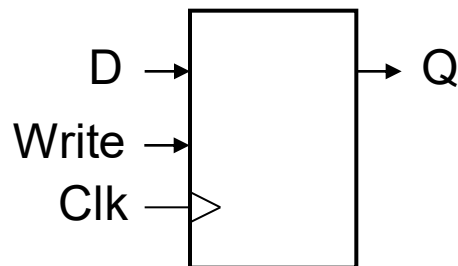
Sequential Elements

- Register: stores data in a circuit
 - Uses a clock signal to determine when to update the stored value
 - Positive edge-triggered: update when Clk changes from 0 to 1



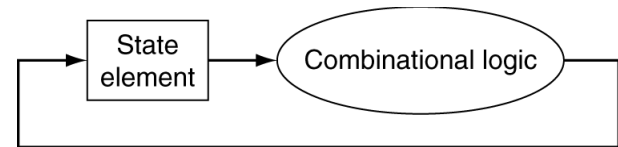
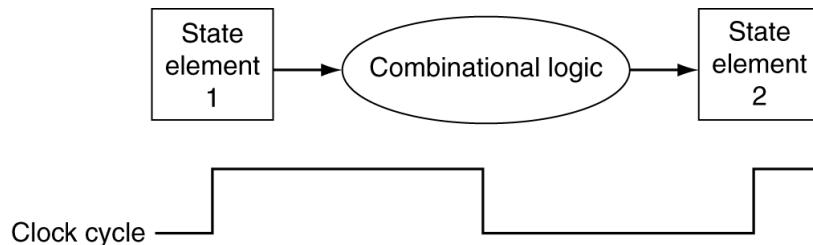
Sequential Elements

- Register with write control
 - Only updates on clock edge when write control input is 1
 - Used when not updated every cycle



Clocking Methodology

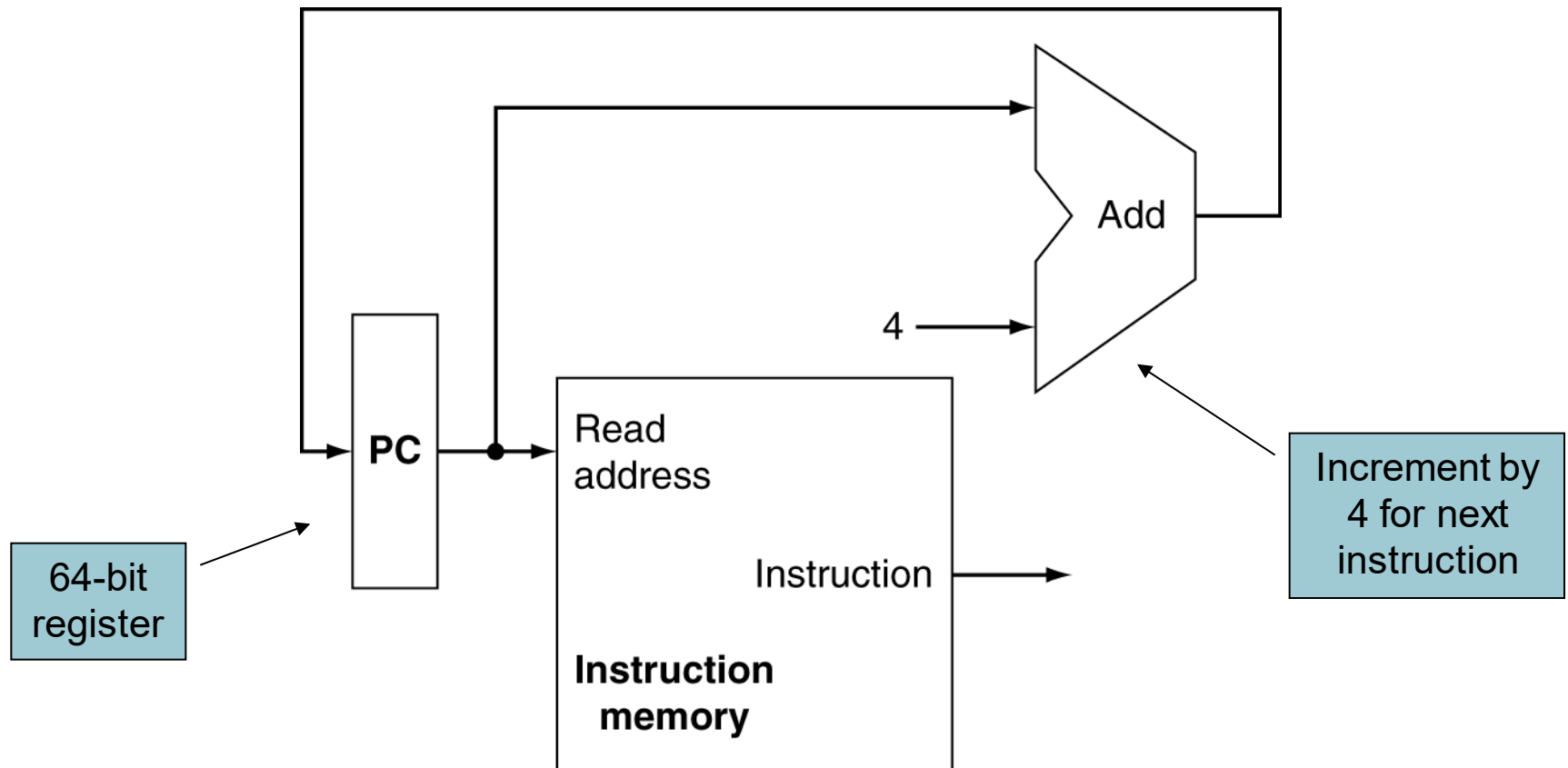
- Combinational logic transforms data during clock cycles
 - Between clock edges
 - Input from state elements, output to state element
 - Longest delay determines clock period



Building a Datapath

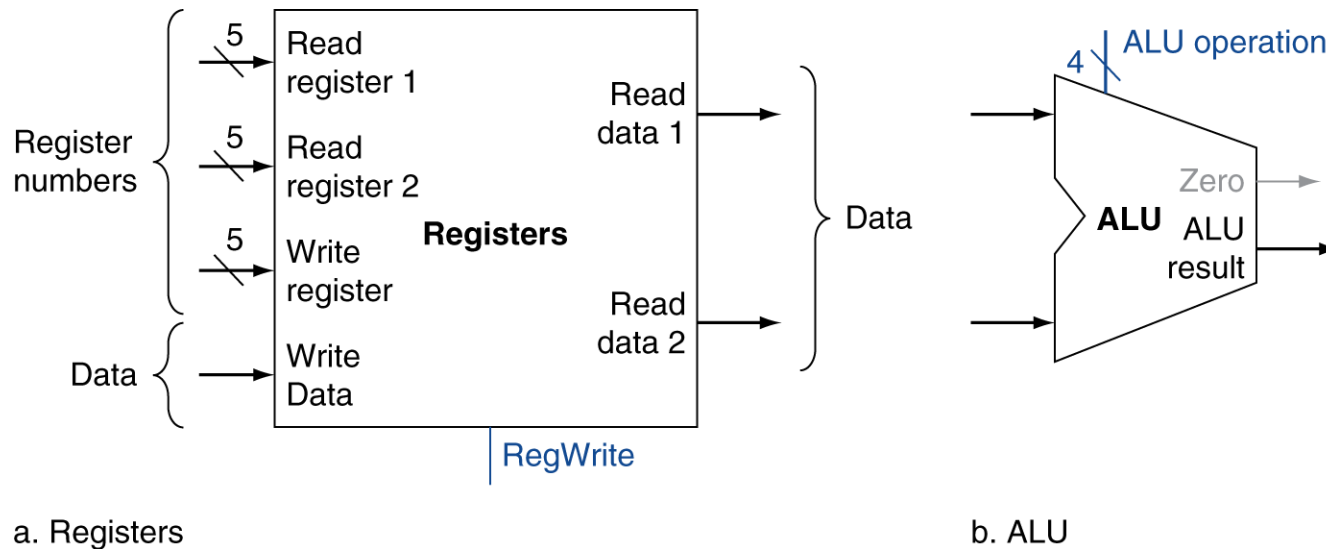
- Datapath
 - Elements that process or hold data in the processor
 - Registers, ALUs, mux's, memories, ...
- Will build a RISC-V datapath incrementally

Instruction Fetch



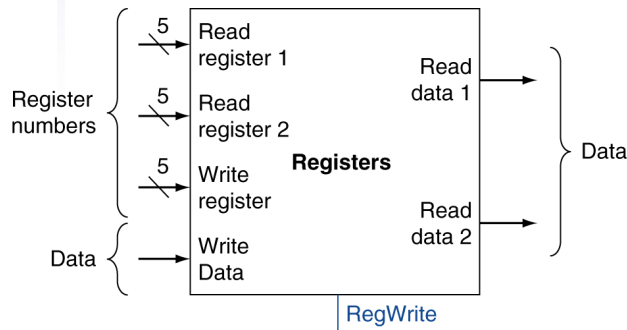
R-Format Instructions

- Read two register operands
- Perform arithmetic/logical operation
- Write register result

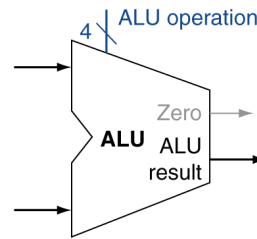


Load/Store Instructions

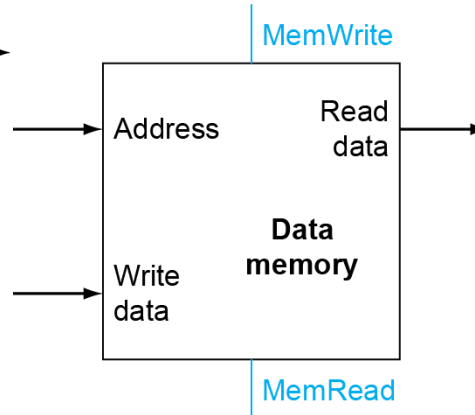
- Read register operands
- Calculate address using 12-bit offset
 - Use ALU, but sign-extend offset
- Load: Read memory and update register
- Store: Write register value to memory



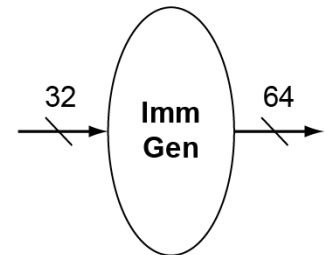
a. Registers



b. ALU



a. Data memory unit

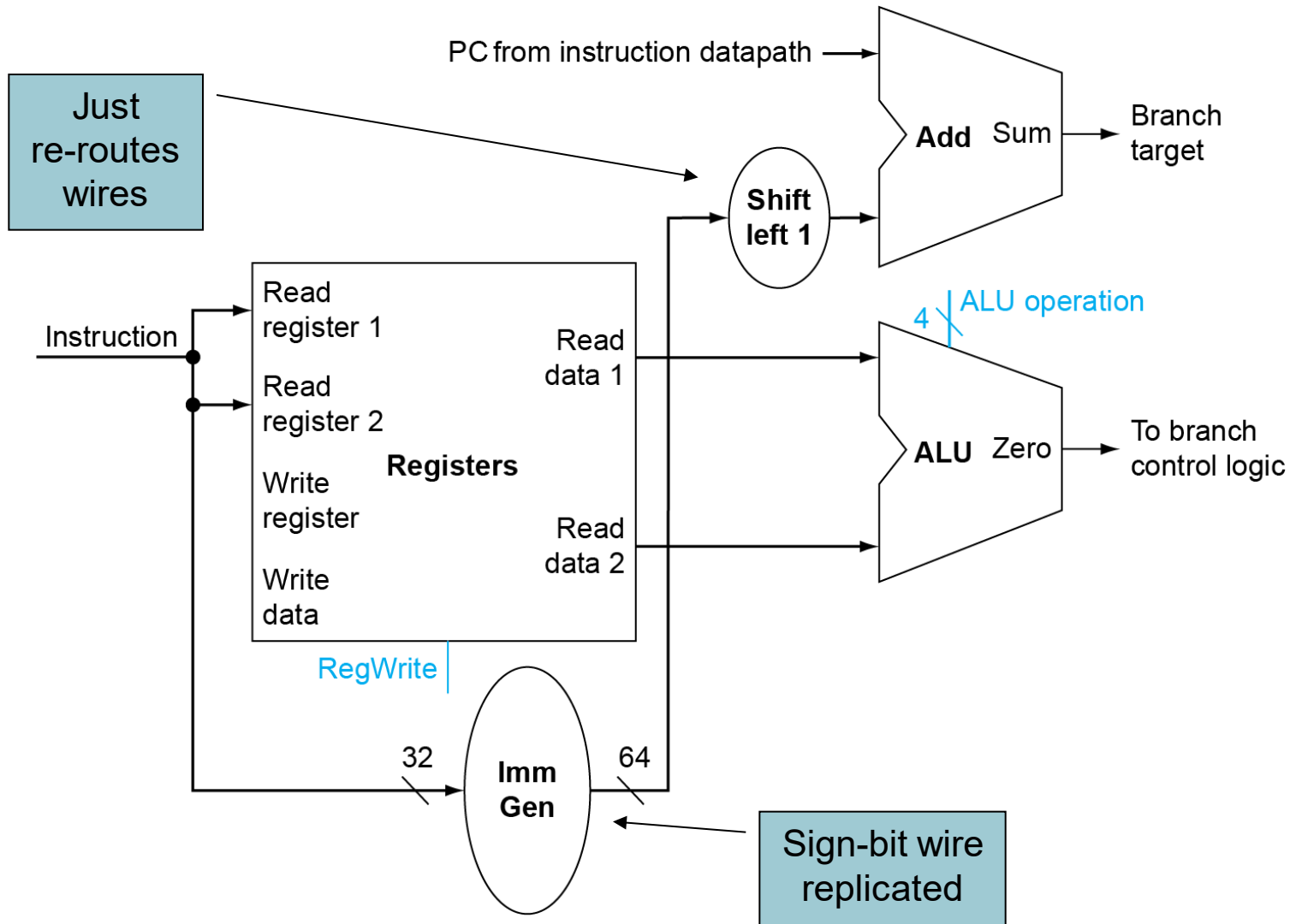


b. Immediate generation unit

Branch Instructions

- Read register operands
- Compare operands
 - Use ALU, subtract and check Zero output
- Calculate target address
 - Sign-extend displacement
 - Shift left 1 bit (halfword displacement)
 - Add to PC value

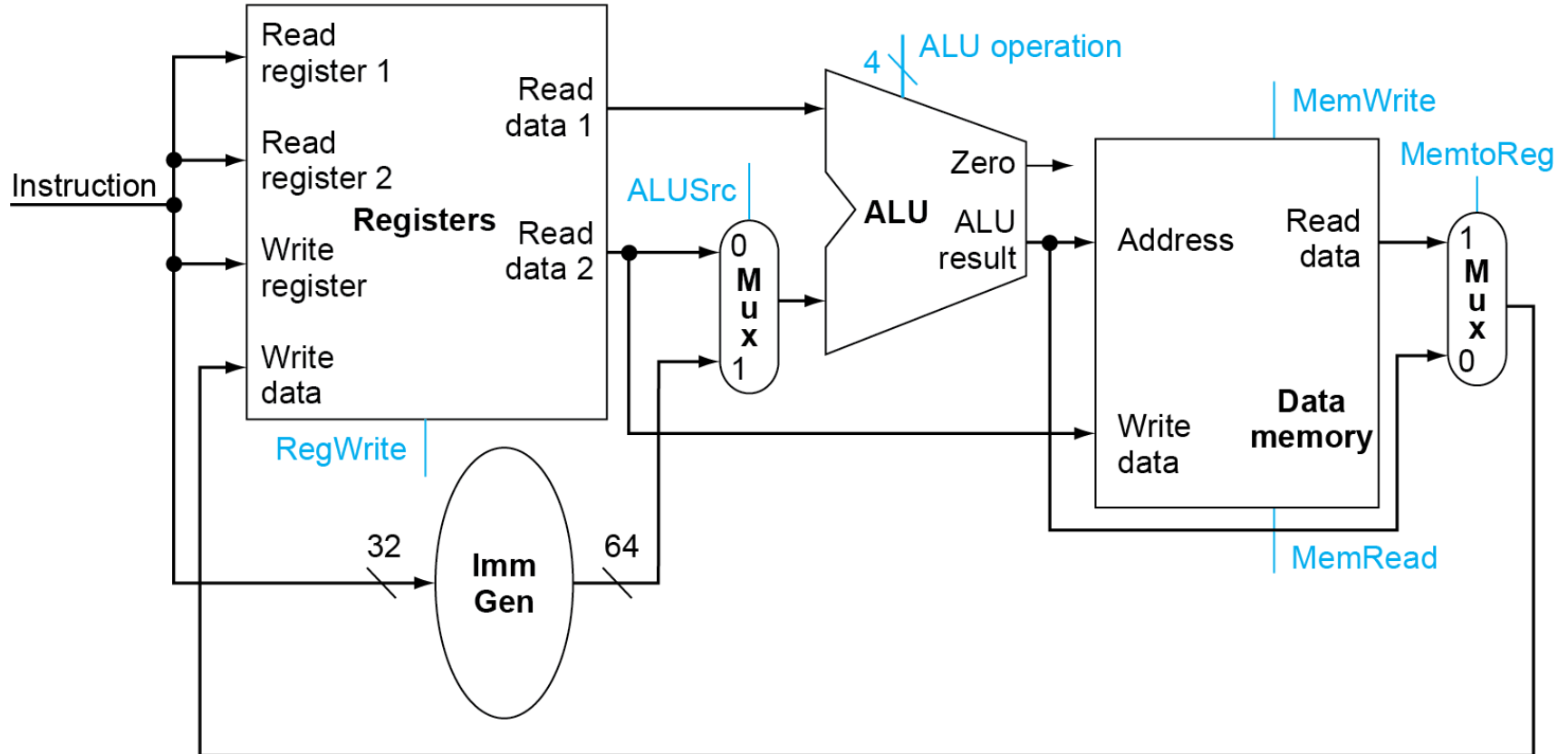
Branch Instructions



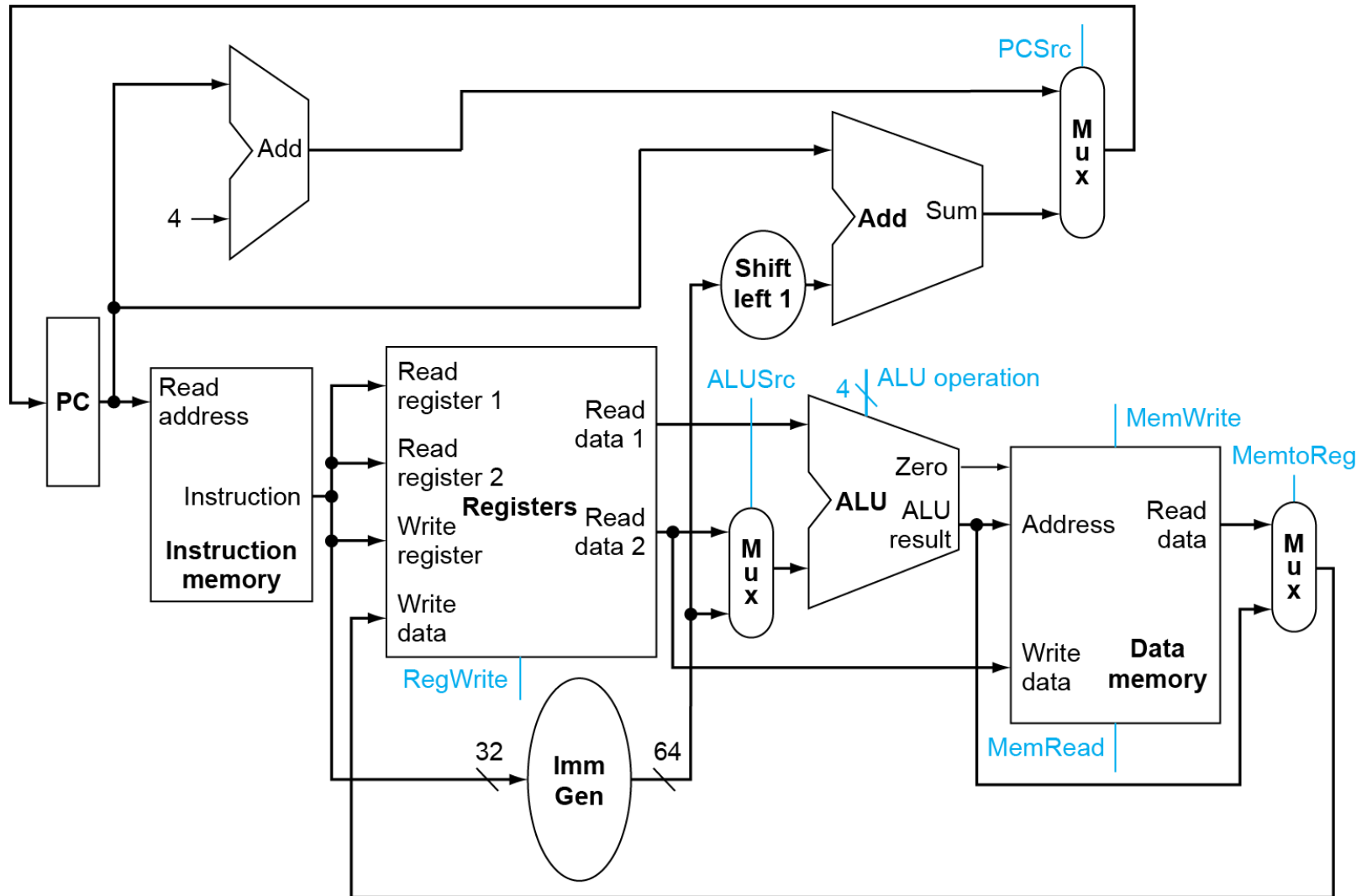
Composing the Elements

- This datapath does an instruction in **one clock cycle**
 - Each datapath element can only do one function at a time
 - Hence, we need separate instruction and data memories
- Use multiplexers where alternate data sources are used for different instructions

R-Type/Load/Store Datapath



Full Datapath with Control Signals



Effect of Control Signals

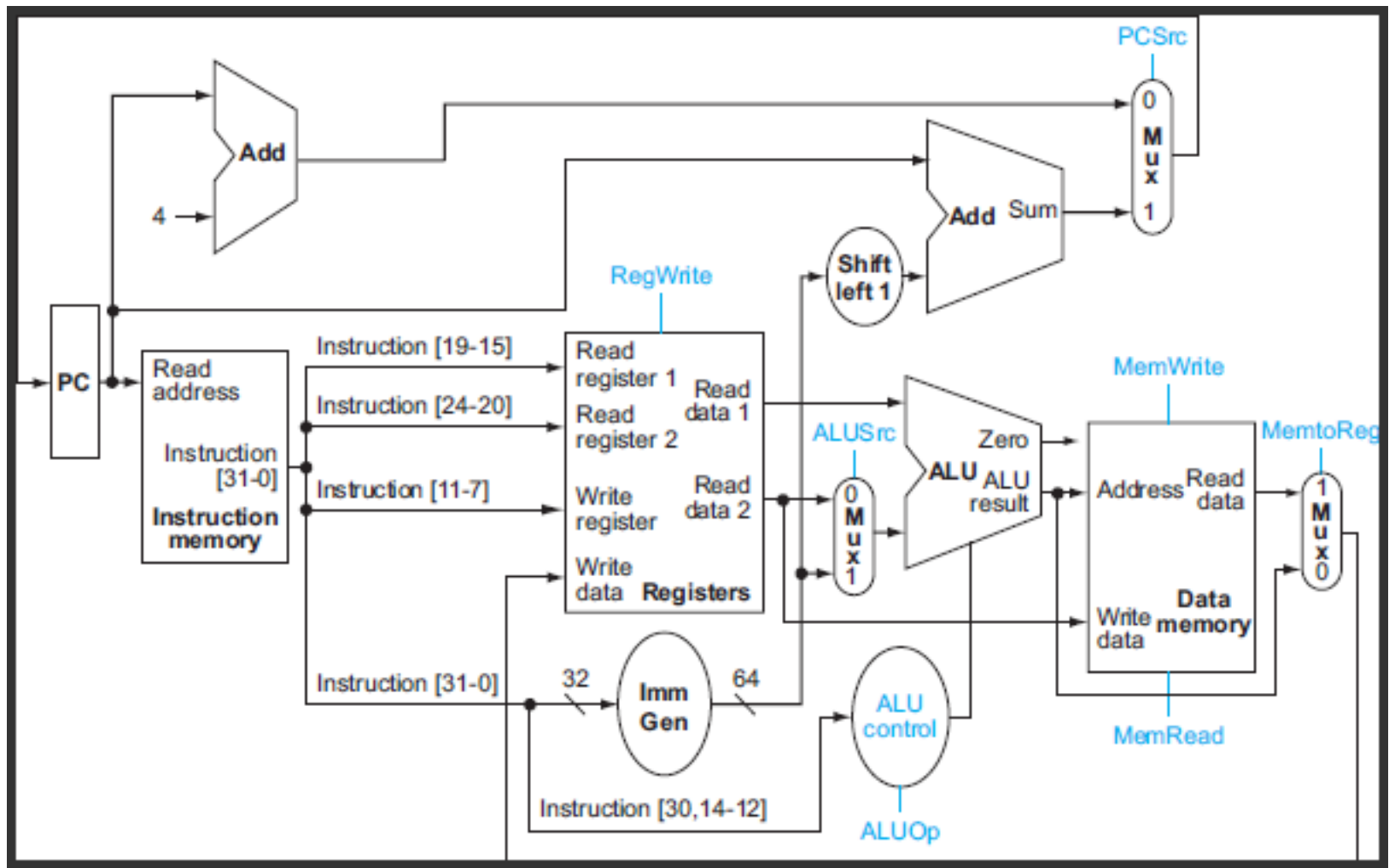
Signal name	Effect when deasserted	Effect when asserted
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, 12 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of $PC + 4$.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

Review of Instruction Formats

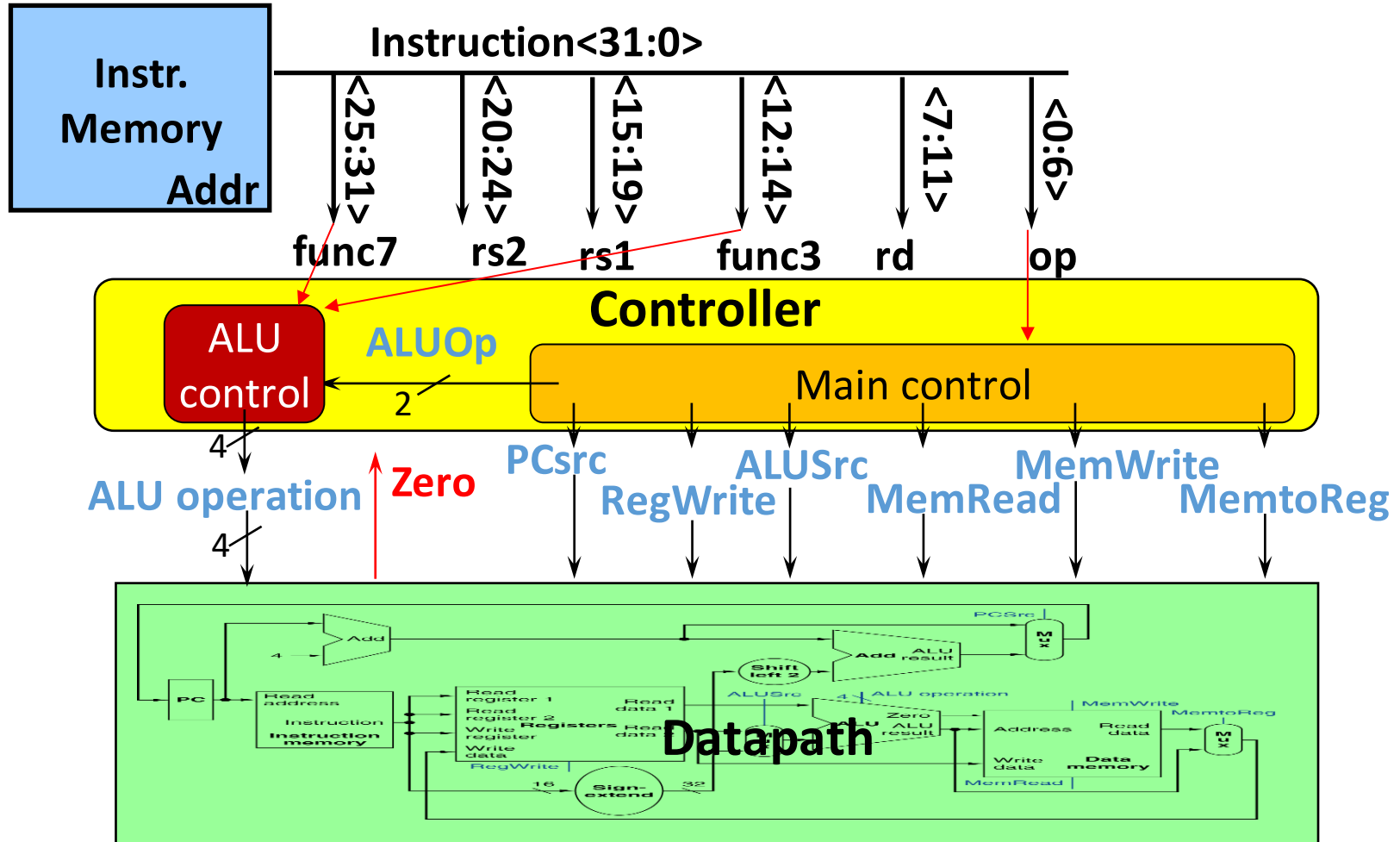
Name (Bit position)	Fields					
	31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

Datapath with Control Signals

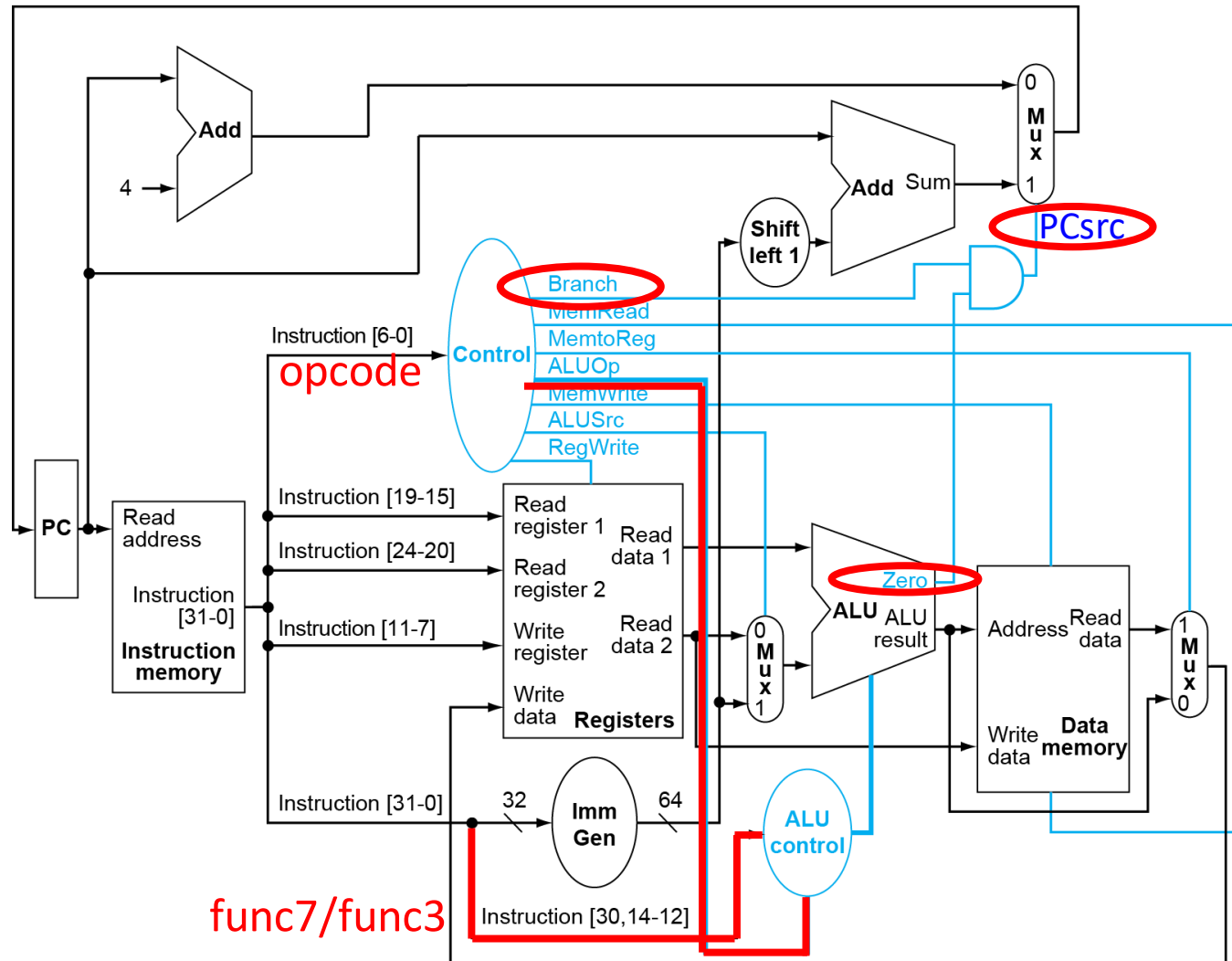
- Compared with the one in page 21



Controller Design



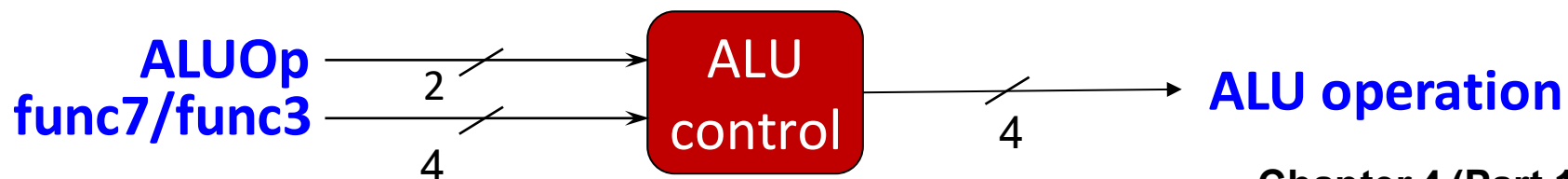
Datapath + Control



ALU Control

- ALU used for
 - Load/Store: add $\rightarrow$ ALUOp = 00
 - Branch: subtract $\rightarrow$ ALUOp = 01
 - R-type: depends on opcode/func $\rightarrow$ ALUOp = 10

ALU operation	Function
0000	AND
0001	OR
0010	add
0110	subtract



ALU Control

- Inputs: 2-bit ALUOp + I[30,14-12]
- ALUOp: derived from opcode

ALU operation

ALU control lines		Function
0000		AND
0001		OR
0010		add
0110		subtract

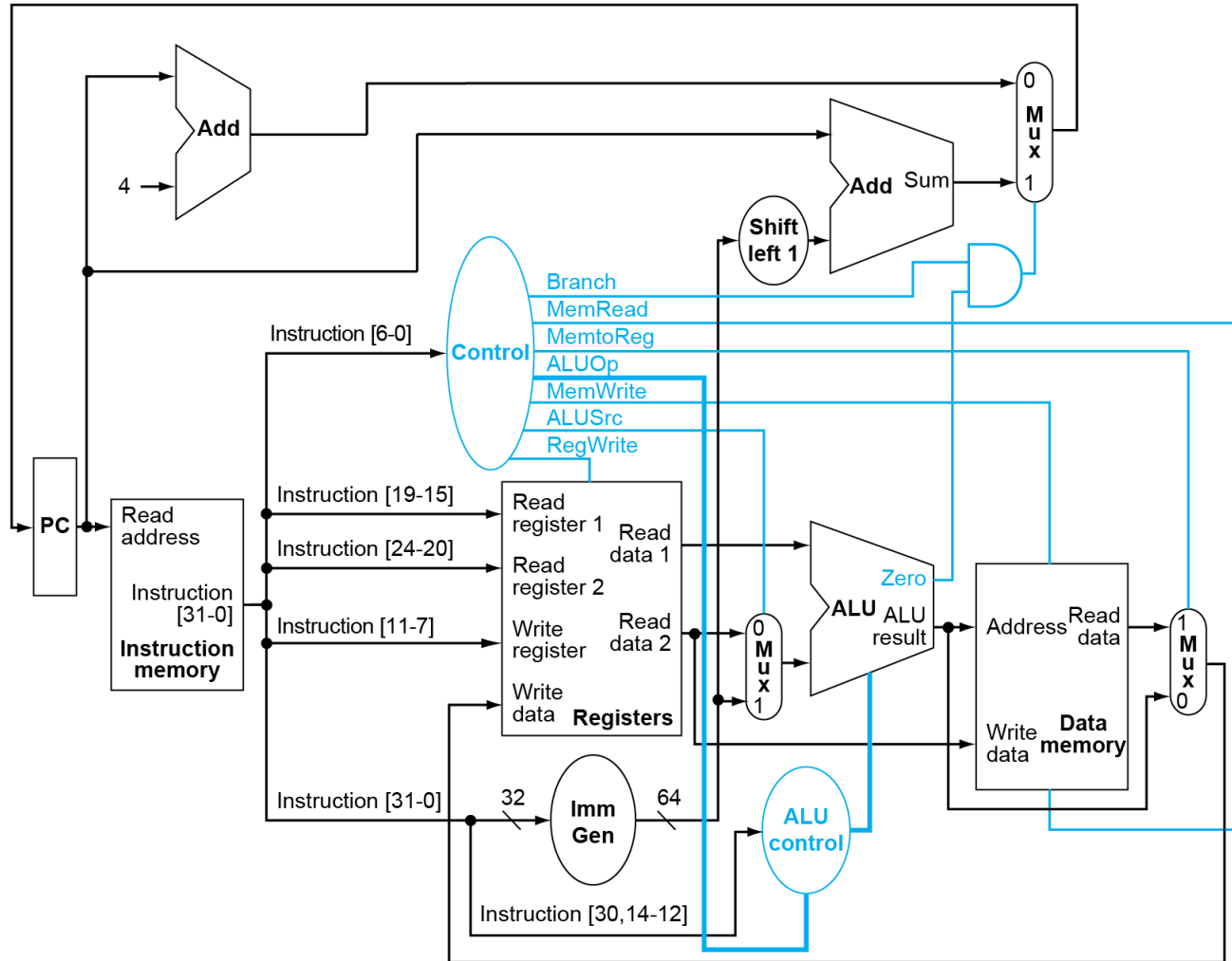
Instruction opcode	ALUOp	operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
ld	00	load doubleword	XXXXXXX	XXX	add	0010
sd	00	store doubleword	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

- The logic design of ALU control can then be derived from the truth table

ALUOp		Funct7 field							Funct3 field			ALU
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	Operation
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

Datapath With Main Control Unit

- Main control unit: derives control signals from instruction



Setting of Control Signals

Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
ld	1	1	1	1	0	0	0	0
sd	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

Truth Table for Main Control Unit

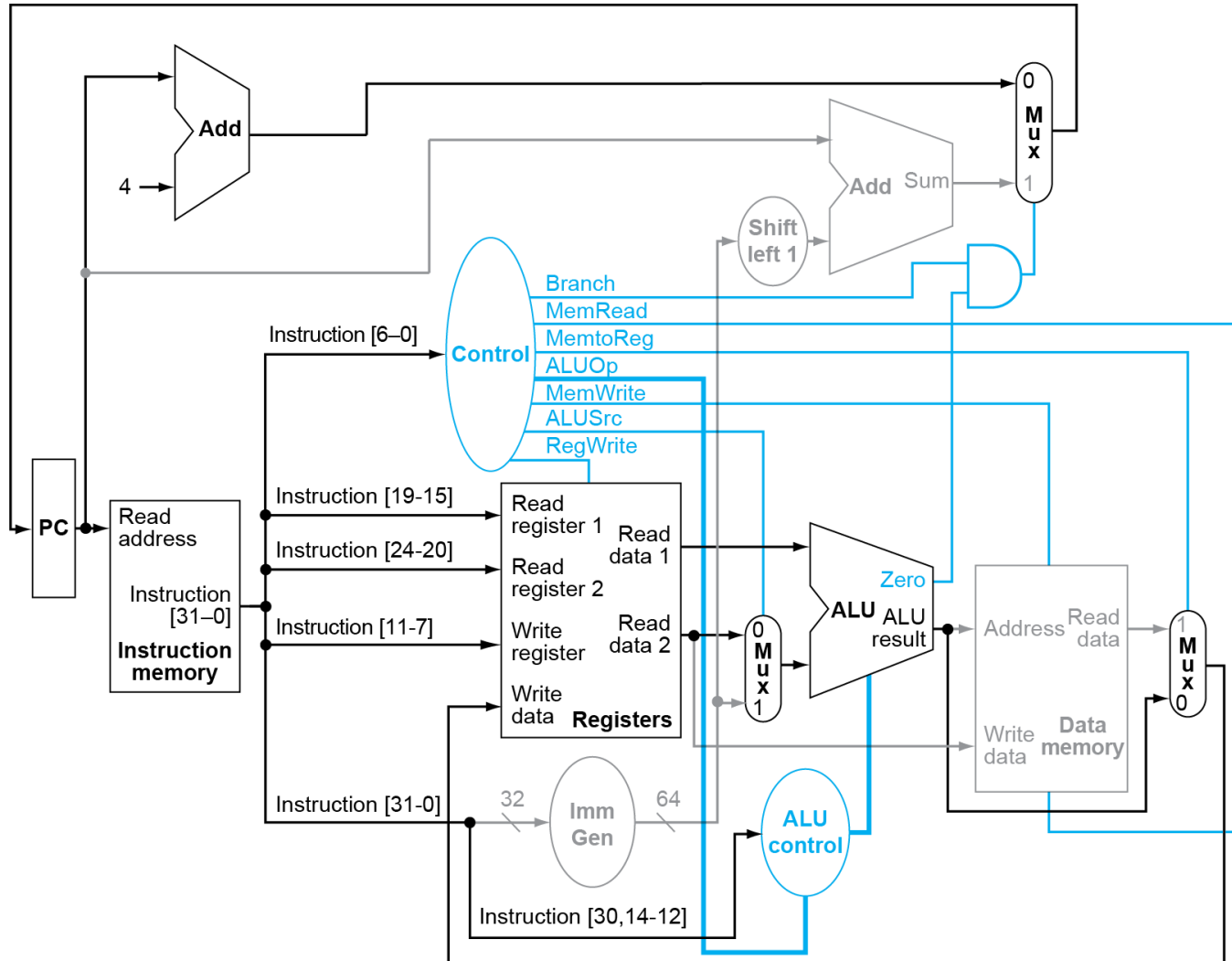
- Opcode (I[6:0]) is used as input
- The logic design of the main control can be derived from the following truth table

Input or output	Signal name	R-format	ld	sd	beq
Inputs	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Outputs	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

Execution for R-Type Instruction

- Example: add x1, x2, x3
 - The instruction is fetched; PC is incremented
 - Two registers, x2 and x3, are read; the main control unit sets the control signals
 - ALU operates on the data read from the register file
 - The result from ALU is written into the destination register x1

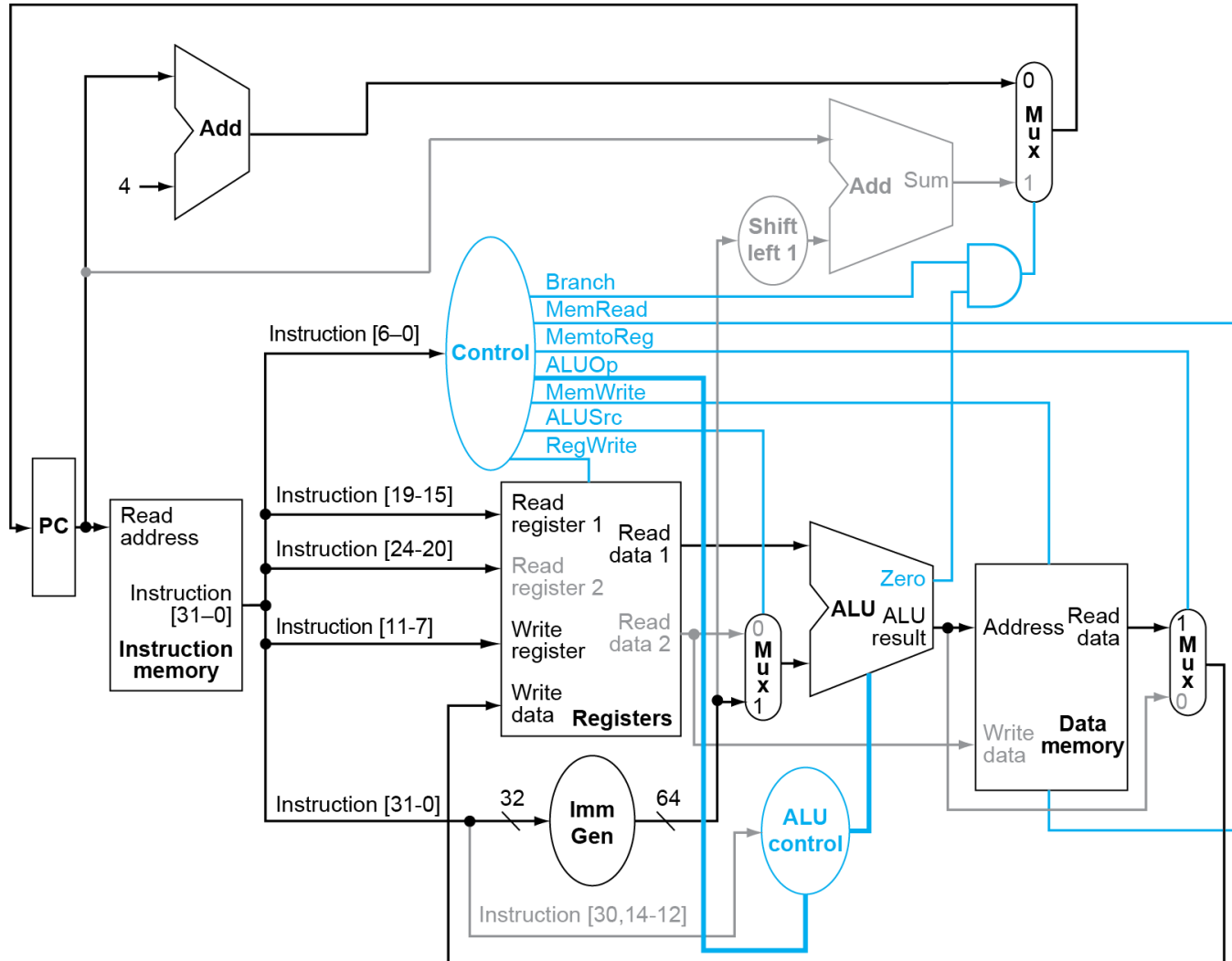
R-Type Instruction



Execution for Load Instruction

- Example: `ld x1, offset(x2)`
 - The instruction is fetched; PC is incremented
 - The value of register x2 is read; 12-bit offset is sign-extended; the main control unit sets the control signals
 - ALU computes the sum of the value read from the register file and the sign-extended offset
 - The result from ALU is used as the address to read the data from the data memory
 - The data from the memory is written into the destination register x1

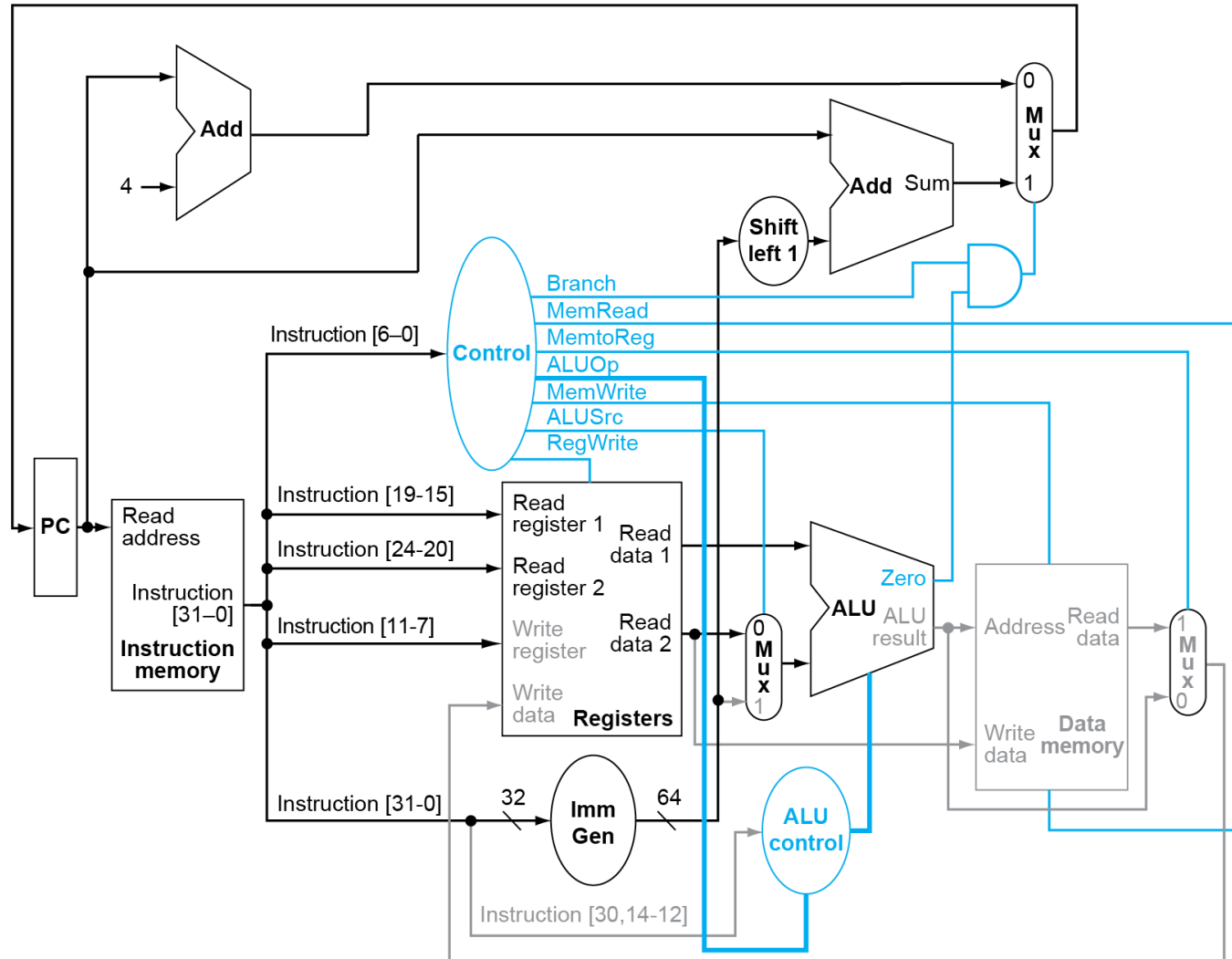
Load Instruction



Execution for Branch Instruction

- Example: `beq x1, x2, offset`
 - The instruction is fetched; PC is incremented
 - Two registers, x1 and x2, are read; 12-bit offset is sign-extended; the main control unit sets the control signals
 - ALU subtracts one data from the other data, both read from the register file; PC is added to the signed-extended, left shifted (by 1 bit) offset to get the target address
 - The Zero from ALU is used to decide which address to store in PC

Branch Instruction



Performance Issue

- Longest delay determines clock period
 - Critical path: load instruction
 - Instruction memory → register file → ALU → data memory → register file
- Not feasible to vary period for different instructions
- Violates design principle
 - Making the common case fast
- Will improve performance by pipelining